

CS101-Quiz2-Review

CS101-Quiz2-Review

Key Points

1. Complexity Analysis

1.1. Landau Symbols

1.2. Code Analysis

1.3. Best-, Worst-, and Average-case

2. Hash Table

2.1. Hash Function Properties

2.2. Dealing with Collisions

Complexity Analysis

Landau Symbols / Notations

$$f(n) = \Theta(g(n)) \quad \exists c_1, c_2, n_0 \in \mathbb{R}^+, \forall n \geq n_0 \quad 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

$$f(n) = O(g(n)) \quad \exists c, n_0 \in \mathbb{R}^+, \forall n \geq n_0 \quad 0 \leq f(n) \leq c \cdot g(n)$$

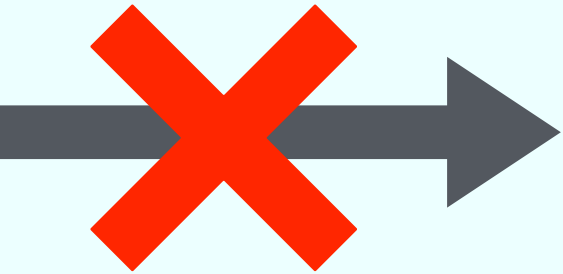
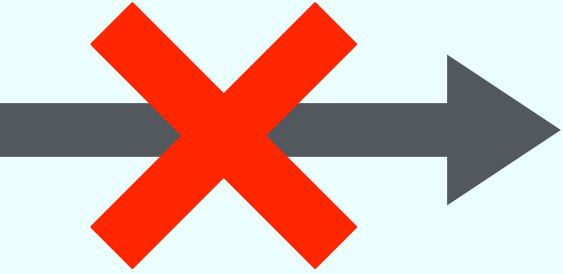
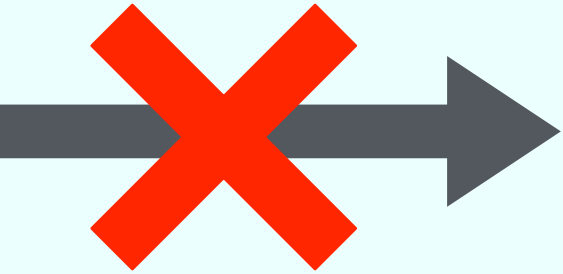
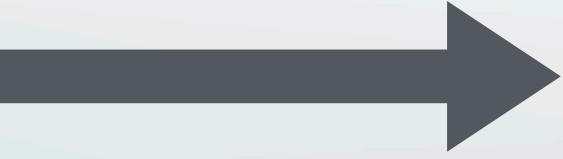
$$f(n) = \Omega(g(n)) \quad \exists c, n_0 \in \mathbb{R}^+, \forall n \geq n_0 \quad 0 \leq c \cdot g(n) \leq f(n)$$

$$f(n) = o(g(n)) \quad \forall c \in \mathbb{R}^+, \exists n_0, \forall n \geq n_0 \quad 0 \leq f(n) < c \cdot g(n)$$

$$f(n) = \omega(g(n)) \quad \forall c \in \mathbb{R}^+, \exists n_0, \forall n \geq n_0 \quad 0 \leq c \cdot g(n) < f(n)$$

Complexity Analysis

Landau Symbols / Notations

$f(n) = \Theta(g(n))$		$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$
$f(n) = O(g(n))$		$0 \leq \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$
$f(n) = \Omega(g(n))$		$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \leq \infty$
$f(n) = o(g(n))$		$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$
$f(n) = \omega(g(n))$		$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$

Complexity Analysis

Landau Symbols / Notations

$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$		$f(n) = \Theta(g(n))$
$0 \leq \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$		$f(n) = O(g(n))$
$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} \leq \infty$	$\longrightarrow$	$f(n) = \Omega(g(n))$
$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$		$f(n) = o(g(n))$
$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$		$f(n) = \omega(g(n))$



Complexity Analysis

Landau Symbols / Notations

$$\log(n!) = \Theta(\log n^n)$$

$$\begin{aligned}\log(n!) &= \log(1) + \log(2) + \dots + \log(n) \\ &\leq \log(n) + \log(n) + \dots + \log(n) \\ &= n \cdot \log(n)\end{aligned}$$

$$\begin{aligned}\log(n!) &= \log(1) + \log(2) + \dots + \log(n) \\ &\geq \log\left(\frac{n}{2}\right) + \log\left(\frac{n}{2} + 1\right) + \dots + \log(n) \\ &\geq \frac{n}{2} \cdot \log\left(\frac{n}{2}\right)\end{aligned}$$

Complexity Analysis

Landau Symbols / Notations

$$1 < \log n < n < n \log n < n^2 < 2^n < 3^n < n! < n^n$$

CS101-Quiz2-Review

Key Points

1. Complexity Analysis

1.1. Landau Symbols

1.2.Code Analysis

1.3.Best-, Worst-, and Average-case

2. Hash Table

2.1.Hash Function Properties

2.2.Dealing with Collisions

Complexity Analysis

Code Analysis



```
1  size_t count = 0;
2  for (int i = 0; i < SIZE; ++i) {
3      for (int j = i; j < SIZE; ++j) {
4          if (array[i] > array[j])
5              ++count;
6      }
7  }
```

Complexity Analysis

Code Analysis

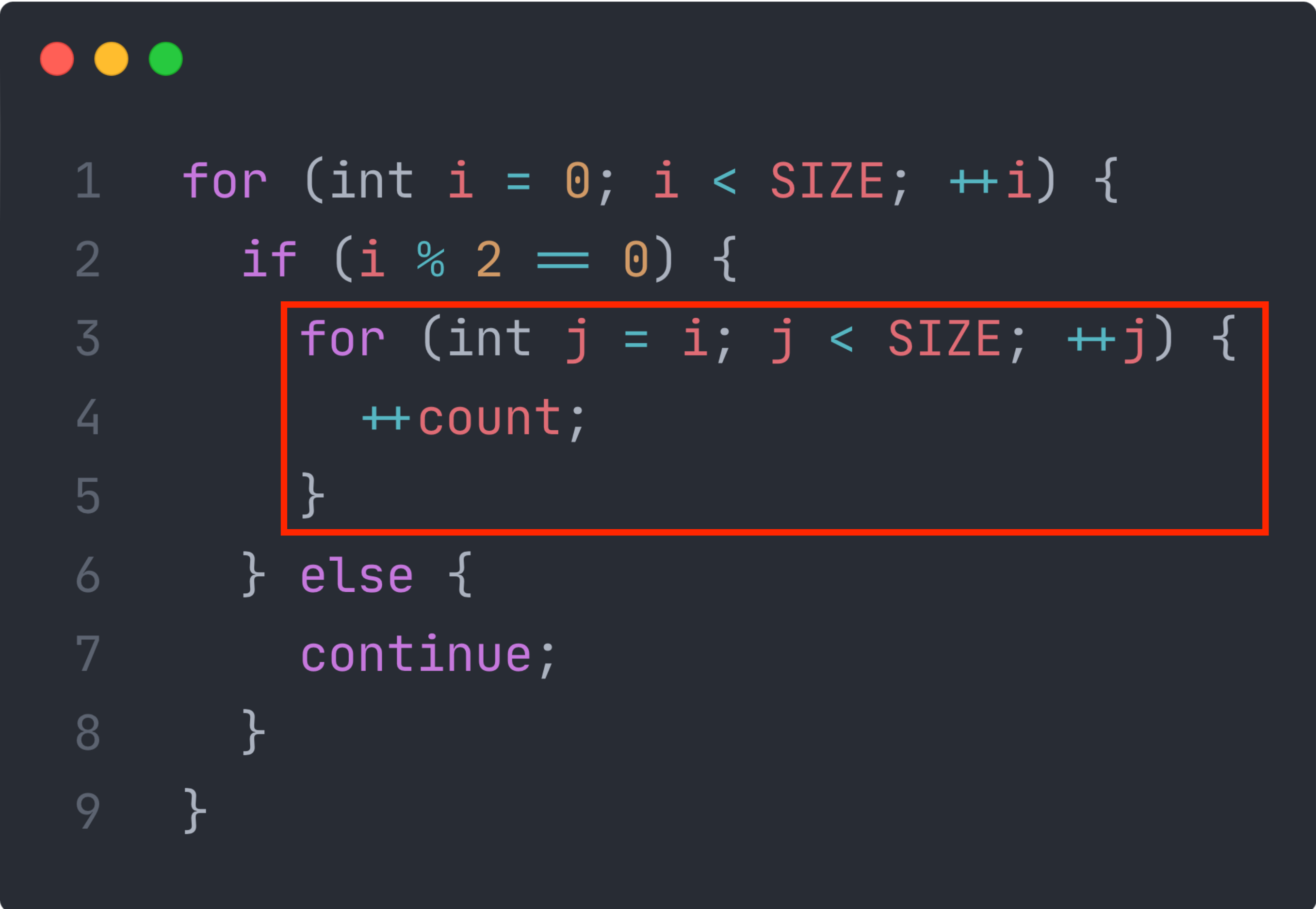


```
1  for (int i = 0; i < strlen(s); ++i) {  
2      putchar(s[i]);  
3  }
```

```
1  size_t len = strlen(s);  
2  for (int i = 0; i < len; ++i) {  
3      putchar(s[i]);  
4  }
```

Complexity Analysis

Code Analysis



```
1  for (int i = 0; i < SIZE; ++i) {  
2      if (i % 2 == 0) {  
3          for (int j = i; j < SIZE; ++j) {  
4              ++count;  
5          }  
6      } else {  
7          continue;  
8      }  
9  }
```


CS101-Quiz2-Review

Key Points

1. Complexity Analysis

1.1. Landau Symbols

1.2. Code Analysis

1.3. Best-, Worst-, and Average-case

2. Hash Table

2.1. Hash Function Properties

2.2. Dealing with Collisions

Complexity Analysis

Best-, Worst-, and Average-case

1. Be aware that “Average” can be ambiguous!

2. Amortized complexity: $\frac{\text{total complexity}}{\text{number of operations}}$

CS101-Quiz2-Review

Key Points

1. Complexity Analysis

1.1. Landau Symbols

1.2. Code Analysis

1.3. Best-, Worst-, and Average-case

2. Hash Table

2.1. Hash Function Properties

2.2. Dealing with Collisions

Hash Table

Hash Function **Restrictions**

1. Deterministic — Same object hash to same value.
2. Equal objects hash to equal value.

Hash Table

Hash Function **Restrictions** / **Good Properties**

1. Deterministic — Same object hash to same value
2. Equal objects hash to equal value
3. Efficiency — Easy to calculate
4. Uniformity — Less collisions in hash tables

Hash Table

Hash Function

```
1 unsigned value = 0;
2 for (int i = 0; i < len; ++i)
3     value = value * base + s[i];
4 return value;
```



Hash Table

Hash Function



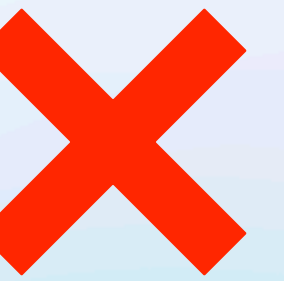
```
1 size_t hash(const char *s) {  
2     return 1234;  
3 }
```



Hash Table

Hash Function

```
1 unsigned value = 0;
2 for (int i = 0; i < len; ++i)
3     value = value * base + s[i];
4 return value + rand() % 1009;
```



CS101-Quiz2-Review

Key Points

1. Complexity Analysis

1.1. Landau Symbols

1.2. Code Analysis

1.3. Best-, Worst-, and Average-case

2. Hash Table

2.1. Hash Function Properties

2.2. Dealing with Collisions

Hash Table

Dealing with Collisions

1. Load Factor $\lambda = \frac{n}{M}$ The average number of entries per "bin".
2. The probability of at least one collision: $1 - \frac{M(M-1)\dots(M-n+1)}{M^n}$

Hash Table

Dealing with Collisions

1. Chained hash table

2. Open Addressing

2.1. Linear Probing

$$h(k, i) = (h'(k) + i) \bmod m$$

2.2. Quadratic Probing

$$h(k, i) = (h'(k) + c_1i + c_2i^2) \bmod m$$

2.3. Double Hashing

$$h(k, i) = (h_1(k) + ih_2(k)) \bmod m$$